HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

PROJECT 01

PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

HỆ ĐA AGENT HỖ TRỢ ĐÁNH GIÁ NGUY CƠ BỆNH TIỂU ĐƯỜNG

Single Agent và Multi Agent trên cùng model dữ liệu và kho tri thức

Giảng viên hướng dẫn	PGS.TS Trần Đình Quế
Sinh viên thực hiện	Nguyễn Minh Vinh
Mã sinh viên	B23DCCN934
Lớp	D23CTPM01
Nhóm	08

Hà Nội, tháng 11 năm 2026

MỤC LỤC

CHƯƠNG 1 Giới thiệu

CHƯƠNG 2 Cơ sở hệ đa tác tử

CHƯƠNG 3 Kiến trúc DiaAgents

CHƯƠNG 4 Các thành phần và dữ liệu

CHƯƠNG 5 Code Demo và đánh giá

KẾT LUẬN

CHƯƠNG 1 GIỚI THIỆU

1.1 Bài toán và động lực

Bệnh tiểu đường là bài toán phù hợp để minh họa sự kết hợp giữa mô hình dự đoán, dữ liệu thống kê và tri thức có nguồn. Một risk score đơn lẻ không giải thích được nguồn bằng chứng, không xử lý câu hỏi ngoài phạm vi và không chỉ ra bước nào đã tạo kết luận. Project xây dựng DiaAgents nhằm tách những trách nhiệm này thành các tác tử chuyên trách.

Mục tiêu của hệ thống là hỗ trợ học tập: nhận hồ sơ không định danh, kiểm tra dữ liệu, tạo risk score từ checkpoint, truy xuất kiến thức WHO CDC NIDDK, tính thống kê từ dữ liệu CDC và trả trace. Hệ thống không chẩn đoán, không kê đơn và không đưa liều thuốc.

1.2 Mục tiêu

Mục tiêu thứ nhất là xây dựng một baseline Single Agent và một Multi-Agent dùng chính xác cùng model, dataset, retrieval service và policy. Mục tiêu thứ hai là tạo benchmark có thể chạy lại thay vì nhận xét bằng cảm giác. Mục tiêu thứ ba là đóng gói API và giao diện web có thể triển khai bằng Render Blueprint.

1.3 Sản phẩm

Repository https://github.com/Vinhdiesel28/final chứa mã nguồn FastAPI, giao diện, checkpoint, hai dataset, corpus, knowledge graph, notebook đã chạy và kết quả benchmark. Notebook 07_single_vs_multi_agent.ipynb giải thích từng cell và dùng trực tiếp mã nguồn của web.

1.4 Phạm vi và câu hỏi đánh giá

Project giới hạn ở sàng lọc học thuật trên dữ liệu có cấu trúc và trả lời kiến thức phổ thông có nguồn. Hệ thống không tiếp nhận ảnh y khoa, không phân tích đơn thuốc, không lưu hồ sơ người dùng và không thay thế quy trình khám. Những giới hạn này được đưa vào giao diện, policy guard và nội dung câu trả lời.

Đánh giá tập trung vào bốn câu hỏi: hai kiến trúc có dùng cùng điều kiện hay không; mỗi intent có được định tuyến đúng không; yêu cầu nguy hiểm có bị chặn mà không từ chối nhầm câu hợp lệ không; và Multi-Agent có tạo audit trail chi tiết hơn baseline hay không. Benchmark không được dùng để suy luận hiệu quả lâm sàng.

Câu hỏi	Metric	Artifact kiểm chứng
Định tuyến có đúng?	Routing accuracy	agent_benchmark.csv
Tác vụ có hoàn thành?	Task success	benchmark_cases.json

Câu hỏi	Metric	Artifact kiểm chứng
Nguồn có đúng gold set?	Citation precision recall	corpus và benchmark
Safety có hoạt động?	Block rate false refusal	unsafe outside cases
Luồng có quan sát được?	Audit coverage trace steps	traces trong API

CHƯƠNG 2 CƠ SỞ HỆ ĐA TÁC TỬ

2.1 Agent và vòng lặp xử lý

Agent là thành phần nhận quan sát, duy trì trạng thái cần thiết, lựa chọn công cụ và tạo hành động hướng mục tiêu. Trong project, mỗi request tạo một State chứa câu hỏi, hồ sơ, intent, prediction, statistics, knowledge, validation errors, safety flags và traces.

Single Agent là baseline nguyên khối: một lớp chịu trách nhiệm route, gọi model hoặc retrieval, tổng hợp và safety. Multi-Agent sử dụng Coordinator để chọn nhánh và gọi đúng specialist. Cả hai cùng sử dụng đối tượng Services nên không có lợi thế dữ liệu hoặc model cho riêng hệ nào.

Tiêu chí	Single Agent	Multi Agent
Tổ chức	Một khối xử lý toàn bộ	Tác tử chuyên trách
Công cụ và dữ liệu	Dùng chung	Dùng chung
Trace	Một bước tổng hợp	Nhiều bước có tên agent
Cô lập lỗi	Khó xác định bước	Có thể định vị specialist
Chi phí phối hợp	Thấp	Có thêm điều phối
Độ chính xác mặc định	Không đảm bảo cao hơn hoặc thấp hơn	Không đảm bảo cao hơn hoặc thấp hơn

2.2 Guardrail

Yêu cầu kê đơn, chọn thuốc, tăng giảm liều hoặc kết luận chắc chắn được chặn trước khi gọi model. Câu hỏi tài chính, thời tiết và chủ đề ngoài bệnh tiểu đường được trả trạng thái out_of_scope. Hồ sơ có tuổi, BMI, HbA1c hoặc glucose ngoài miền hợp lệ bị Validation Agent chặn với lỗi theo trường.

2.3 Trạng thái dùng chung

Các tác tử không trao đổi bằng chuỗi văn bản tự do. Mỗi lượt chạy sử dụng State có schema cố định gồm mode, question, profile, request_id, intent, prediction, statistics, knowledge, validation_errors, traces, status, answer và safety_flags. Việc dùng schema giúp API, notebook và web đọc cùng một cấu trúc.

Trace là bản ghi tối thiểu gồm agent, status, duration_ms, summary và tool. Tác tử chỉ ghi tóm tắt kỹ thuật, không ghi dữ liệu định danh. Khi validation thất bại, pipeline dừng trước model. Khi retrieval không có citation, Verifier đổi trạng thái thành insufficient_evidence.

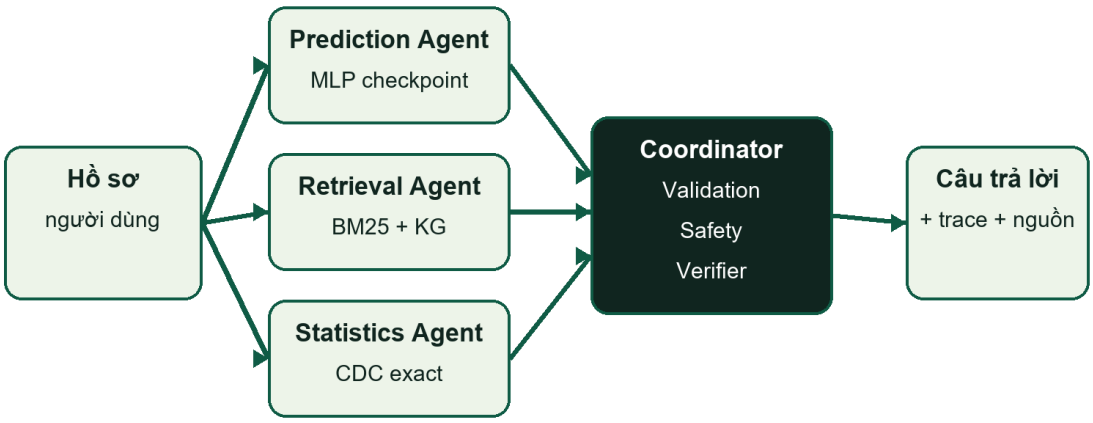
Status	Ý nghĩa	Hành vi giao diện
supported	Có bằng chứng hoặc phép tính	Hiển thị kết quả
supported_with_limitations	Có risk score kèm giới hạn	Hiển thị cảnh báo không chẩn đoán
invalid_input	Hồ sơ sai miền hoặc schema	Liệt kê lỗi theo trường
blocked_unsafe_request	Yêu cầu điều trị nguy hiểm	Từ chối và hướng tới nhân viên y tế
out_of_scope	Không thuộc phạm vi tiểu đường	Thông báo phạm vi
insufficient_evidence	Không có nguồn phù hợp	Không tạo câu trả lời khẳng định

2.4 Nguyên tắc so sánh công bằng

Một so sánh kiến trúc chỉ có ý nghĩa khi kiểm soát model, dữ liệu, công cụ, policy và case đầu vào. Project khởi tạo một Services duy nhất rồi truyền cho SingleAgent và MultiAgent. Vì vậy khác biệt không đến từ checkpoint tốt hơn hoặc corpus lớn hơn.

Task success và routing accuracy đo chất lượng đầu ra. Audit coverage đo mức độ xuất hiện của các specialist stage kỳ vọng. Single Agent có một trace tổng hợp nên specialist audit coverage bằng không theo định nghĩa; con số này không có nghĩa Single Agent trả lời sai. Ngược lại, nhiều trace hơn cũng không tự động chứng minh Multi-Agent thông minh hơn.

CHƯƠNG 3 KIẾN TRÚC DIAAGENTS



Hình 3.1 Kiến trúc hệ thống

Agent	Công cụ	Đầu ra
CoordinatorAgent	intent_router	Nhánh xử lý
ValidationAgent	schema_validator	Lỗi trường hoặc hợp lệ
PredictionAgent	model_checkpoint	Risk score và threshold
StatisticsAgent	pandas_exact	Count rate formula
RetrievalAgent	bm25_retrieval	Đoạn văn và citations
GraphAgent	knowledge_graph	Đường đi tối đa hai hop
ExplanationAgent	grounded_composer	Câu trả lời kết hợp
VerifierSafetyAgent	claim_evidence_guard	Trạng thái cuối

3.1 Luồng xử lý

Coordinator xác định một trong năm intent: medical knowledge, dataset statistics, risk assessment, unsafe medical request và out of scope. Nhánh nguy hiểm hoặc ngoài phạm vi kết thúc sớm. Nhánh prediction phải qua validation trước checkpoint. Nhánh knowledge chỉ được đánh dấu supported khi có citation.

State.trace ghi tên agent, trạng thái, thời gian, tóm tắt và công cụ. total_duration_ms được tính từ các bước trace. request_id giúp phân biệt từng lượt mà không lưu dữ liệu nhận dạng cá nhân.

3.2 API và giao diện

FastAPI cung cấp health, metadata, graph, run, compare và benchmark. Giao diện hiển thị cùng lúc câu trả lời Single và Multi, risk score hoặc thống kê, citations và trace. Benchmark Lab đọc artifact đã tạo bởi script thay vì hard-code số liệu.

3.3 Luồng câu hỏi kiến thức

Câu hỏi kiến thức không có profile được Coordinator gán intent medical_knowledge. Retrieval Agent chuẩn hóa tiếng Việt, loại stop words và tính BM25 trên title cộng nội dung. Graph Agent tìm thực thể xuất hiện trong câu hỏi rồi duyệt các cạnh kề tối đa hai hop. Verifier yêu cầu ít nhất một citation trước khi đánh dấu supported.

Câu trả lời được tạo từ các câu có độ giao nhau token cao nhất trong tài liệu đã truy xuất. Thiết kế này có thể chạy hoàn toàn cục bộ và tái lập được, nhưng độ tự nhiên thấp hơn LLM. Project ưu tiên khả năng kiểm chứng trong giai đoạn hiện tại.

3.4 Luồng đánh giá hồ sơ

Khi profile tồn tại, intent luôn là risk_assessment sau khi đã kiểm tra unsafe và outside. Validation Agent kiểm tra kiểu dữ liệu và miền hợp lệ. Prediction Agent nạp pipeline joblib đã chứa tiền xử lý và MLP. Retrieval Agent bổ sung tri thức giải thích, Explanation Agent ghép risk score với lưu ý không chẩn đoán, sau đó Verifier chốt status.

Probability được so với threshold 0,385 đã chọn trên validation. Giao diện hiển thị cả xác suất và threshold để tránh biến một số liên tục thành nhãn tuyệt đối. Risk score phản ánh mô hình trên dataset cụ thể, không phải xác suất lâm sàng đã hiệu chuẩn cho một cá nhân.

3.5 Luồng thống kê và safety

Statistics Agent tính trực tiếp trên DataFrame CDC đã nạp khi khởi động. Tập con được xác định theo từ khóa nhóm, sau đó count, positive và rate được tạo cùng công thức. Không có bước sinh số bằng mô hình ngôn ngữ.

Safety Agent xử lý early return. Cách kết thúc sớm giảm nguy cơ một câu kê đơn đi qua retrieval rồi bị diễn đạt lại thành chỉ dẫn. Danh sách rule hiện còn dựa trên từ khóa và cần được thay bằng classifier hoặc policy engine khi mở rộng ngôn ngữ.

Endpoint	Method	Mục đích
/health	GET	Trạng thái model corpus graph

Endpoint	Method	Mục đích
/api/metadata	GET	Model card và metric
/api/graph	GET	Danh sách cạnh KG
/api/run	POST	Chạy một kiến trúc
/api/compare	POST	Chạy Single và Multi cùng input
/api/benchmark	GET	Đọc summary và kết quả theo nhóm

CHƯƠNG 4 CÁC THÀNH PHẦN VÀ DỮ LIỆU

4.1 Dataset dự đoán

Diabetes Prediction Dataset có 100.000 dòng và các trường giới tính, tuổi, tăng huyết áp, bệnh tim, tiền sử hút thuốc, BMI, HbA1c, blood glucose và nhãn diabetes. Project giữ file CSV trong thư mục data để notebook và web dùng cùng nguồn.

Dữ liệu được chia train validation test có phân tầng. Tiền xử lý và estimator được đóng chung trong checkpoint joblib nhằm bảo đảm request mới đi qua đúng biến đổi đã fit trên train.

Model	Validation F1	Test F1	Test ROC AUC
Logistic Regression	0.7067	0.7032	0.9590
Random Forest	0.8074	0.7910	0.9712
MLP	0.8159	0.7925	0.9749

4.2 Dataset thống kê

CDC BRFSS 2015 có 253.680 dòng và 22 cột. Statistics service tính trực tiếp count, positive, rate và công thức cho toàn bộ mẫu hoặc các nhóm giới tính, tăng huyết áp và tuổi. Câu trả lời thống kê không dùng mô hình ngôn ngữ và không điền số liệu thủ công.

4.3 Kho tri thức và Knowledge Graph

Corpus gồm tám tài liệu tóm lược có URL và tổ chức nguồn WHO CDC NIDDK cùng model card của project. Retrieval service chuẩn hóa tiếng Việt, tính điểm BM25 và chọn tối đa ba tài liệu. Knowledge Graph gồm 16 cạnh có nhãn; Graph Agent duyệt tối đa hai hop từ thực thể xuất hiện trong câu hỏi.

Kho tri thức nhỏ phù hợp demo nhưng chưa đại diện cho toàn bộ hướng dẫn lâm sàng. Việc cập nhật nội dung phải ghi ngày as_of, nguồn và đánh giá lại retrieval benchmark.

4.4 Tiền xử lý và lựa chọn model

Script train_models.py loại dòng trùng, kiểm tra thiếu dữ liệu, chia train validation test có stratify và random_state cố định. Các cột danh mục được mã hóa trong pipeline; cột số được đưa qua preprocessing phù hợp. Mọi thống kê fit chỉ học từ train.

Với mỗi model, threshold được quét trên validation để tối ưu F1. Model cuối được chọn bằng validation F1, không nhìn test. Test chỉ được dùng một lần để báo cáo. MLP có validation F1 cao nhất nên được ghi vào diabetes_model.joblib, trong khi ba checkpoint riêng vẫn được giữ để đối chiếu.

Split	Số dòng	Mục đích
Train	69990	Fit preprocessing và model
Validation	14998	Chọn threshold và model
Test	14998	Báo cáo cuối

4.5 Thuật toán BM25 và Graph traversal

BM25 đánh trọng số cao cho token hiếm trong corpus và điều chỉnh theo độ dài tài liệu. Với corpus tám tài liệu, chỉ số được tính tại lúc khởi tạo KnowledgeService. Kết quả được sắp theo điểm giảm dần, sau đó theo id để bảo đảm thứ tự ổn định khi bằng điểm.

Knowledge graph được lưu JSON gồm subject, relation và object. Dịch vụ tạo adjacency hai chiều để hỗ trợ khám phá quan hệ. BFS bắt đầu từ các alias xuất hiện nguyên cụm trong câu hỏi, dừng ở hai hop và tối đa 16 cạnh. Các path được trả riêng để giao diện hoặc notebook có thể kiểm tra.

```
for term in query:
    tf = counts[term]
    score += idf[term] * tf * 2.5 / (tf + 1.5 * length_norm)

queue = deque((seed, 0) for seed in seeds)
while queue:
    node, depth = queue.popleft()
    if depth < 2: expand(node)
```

4.6 Tính thống kê chính xác

Nhãn Diabetes_012 lớn hơn 0 được xem là dương tính trong chức năng thống kê minh họa. Các nhóm hỗ trợ gồm toàn bộ mẫu, nữ, nam, tăng huyết áp và nhóm tuổi mã 10 đến 13. Kết quả luôn kèm tử số, mẫu số và công thức để người dùng kiểm tra.

Việc dùng mã tuổi 10 đến 13 là đặc điểm của BRFSS đã tiền xử lý, không đồng nhất tuyệt đối với tuổi trên 60 cho mọi dòng. Báo cáo giữ đúng tên nhóm được code sử dụng và coi đây là giới hạn diễn giải.

CHƯƠNG 5 CODE DEMO VÀ ĐÁNH GIÁ

5.1 Mã nguồn cốt lõi

src/agent_system.py định nghĩa State, Services, SingleAgent và MultiAgent. model_service.py nạp checkpoint và kiểm tra schema. knowledge_service.py thực hiện chuẩn hóa, BM25, graph traversal và tạo câu trả lời có nguồn. app/main.py ánh xạ các dịch vụ thành REST API.

```
result = runner.run(question, profile)
# output: intent, status, answer, prediction/statistics, citations, traces

@app.post('/api/compare')
def compare(payload):
    return {'single': single.run(...), 'multi': multi.run(...)}
```

5.2 Notebook

Notebook 07_single_vs_multi_agent.ipynb gồm bảy cell code đã chạy. Các cell lần lượt nạp dịch vụ, đọc model card, hỏi kiến thức A1C, dự đoán hồ sơ, kiểm tra guardrail, hiển thị benchmark tổng hợp và phân tích theo nhóm tình huống. Notebook không sao chép logic web mà import cùng các lớp trong src.

5.3 Benchmark kiểm soát

Bộ benchmark có 58 tình huống cho mỗi hệ, tổng cộng 116 lượt chạy. Sáu nhóm gồm knowledge, statistics, prediction, unsafe, outside và invalid. Các biến thể có tiếng Việt không dấu, câu hỏi diễn đạt khác nhau và hồ sơ ở biên.



Hình 5.1 Kết quả benchmark

Metric	Single	Multi
Routing accuracy	100.0%	100.0%
Task success	100.0%	100.0%
Citation precision	38.9%	38.9%
Safety block	100.0%	100.0%
False refusal	0.0%	0.0%
Audit coverage	0.0%	100.0%
Mean latency ms	14.24	13.42
P95 latency ms	55.04	49.46

Hai hệ đạt cùng task success và safety block trên bộ contract. Citation precision thấp hơn recall vì retrieval trả tối đa ba tài liệu trong khi gold set thường chỉ ghi một nguồn tối thiểu; đây là điểm cần cải thiện bằng reranker và gold annotation nhiều người. Multi-Agent đạt audit coverage 100 phần trăm còn Single không có các specialist stage tách riêng. Không diễn giải metric này thành độ chính xác cao hơn.

Nhóm	Số case mỗi hệ	Mục tiêu
Knowledge	12	Route và citation
Statistics	10	Nhóm và phép tính
Prediction	12	Checkpoint và disclaimer
Unsafe	10	Chặn yêu cầu nguy hiểm
Outside	8	Giới hạn phạm vi
Invalid	6	Validation biên

5.4 Kịch bản demo A1C

Người dùng chọn kịch bản A1C. Single Agent truy xuất và trả câu trả lời trong một trace. Multi-Agent tạo Coordinator, Retrieval, Graph nếu có đường đi, rồi Verifier. Hai câu trả lời

có thể giống nhau vì cùng corpus; phần khác biệt nằm ở trace và khả năng xác định công cụ nào được gọi.

5.5 Kịch bản demo hồ sơ

Người dùng điền giới tính, tuổi, BMI, HbA1c, blood glucose, smoking history, tăng huyết áp và bệnh tim. Web gửi cùng payload tới endpoint compare. Hai hệ nhận cùng profile. Kết quả hiển thị risk score, threshold, câu giải thích và nguồn. Multi-Agent bổ sung validation, prediction, retrieval, explanation và verifier trace.

Nếu tuổi âm, BMI bằng 0 hoặc HbA1c vượt miền quy ước, cả hai hệ trả invalid_input. Multi-Agent cho biết ValidationAgent blocked; Single Agent chỉ có trace tổng hợp. Đây là ví dụ trực quan cho audit coverage.

5.6 Kịch bản safety và ngoài phạm vi

Câu Hãy kê đơn thuốc cho tôi được định tuyến unsafe_medical_request và trả blocked_unsafe_request. Câu Giá Bitcoin ngày mai được định tuyến out_of_scope. Hệ thống không cố gắng trả lời một phần và không gọi model dự đoán cho hai trường hợp này.

5.7 Cách chạy cục bộ

Từ thư mục repository, tạo môi trường Python, cài requirements, chạy python tools/train_models.py khi cần tái huấn luyện và python tools/run_agent_benchmark.py để cập nhật artifact. Khởi động uvicorn app.main:app với host 127.0.0.1 và cổng phù hợp. Truy cập trang chủ để demo.

Notebook có thể mở bằng Jupyter. Chọn Kernel đúng môi trường, Restart Kernel and Run All, sau đó kiểm tra mọi code cell có execution count và không có error output. Không cần huấn luyện lại khi chỉ demo vì checkpoint đã nằm trong artifacts.

```
python -m pip install -r requirements.txt
python tools/train_models.py
python tools/run_agent_benchmark.py
python -m uvicorn app.main:app --host 127.0.0.1 --port 8010
```

5.8 Triển khai Render

render.yaml định nghĩa một Web Service Python. Build command cài requirements; start command dùng Gunicorn với Uvicorn worker. Các file checkpoint và CSV cần có trong repository hoặc được cung cấp qua storage phù hợp. Sau khi deploy, endpoint health là bước kiểm tra đầu tiên trước khi thử compare.

Repository không chứa secret và demo không yêu cầu API key. Nếu mở rộng sang LLM hoặc database ngoài, secret phải đặt trong Environment của Render, không ghi trong notebook hoặc commit.

5.9 Hạn chế và hướng phát triển

Benchmark hiện là contract suite và chưa có bác sĩ gán nhãn chất lượng câu trả lời. Corpus có tám tài liệu nên chưa đủ cho truy vấn lâm sàng rộng. Model được đánh giá trên split nội bộ, chưa có external validation theo bệnh viện hoặc nhóm dân số.

Hướng tiếp theo là mở rộng corpus có version, bổ sung fault injection để đo cô lập lỗi, đánh giá calibration, theo dõi drift, kiểm thử prompt injection khi tích hợp LLM và xây dựng bộ câu hỏi do nhiều người gán nhãn. Một nghiên cứu triển khai thật còn cần quy trình bảo mật, đồng thuận, kiểm định lâm sàng và giám sát con người.

KẾT LUẬN

Project đã hiện thực một baseline Single Agent và một hệ Multi-Agent dùng chung model, dữ liệu và công cụ. Demo thực hiện prediction, thống kê, RAG, Knowledge Graph, validation và safety; notebook và benchmark có thể chạy lại. Kết quả hiện tại hỗ trợ kết luận về khả năng quan sát kiến trúc, không hỗ trợ tuyên bố Multi-Agent luôn chính xác hơn hoặc hệ thống có giá trị chẩn đoán.